

How to use a Source DLL and Ring.dll to generate uniform ray rings in non-sequential mode

Authored by

David Keith

Abstract

This article provides an overview of the Ring.dll functionality. While random sources with a large number of rays are generally best for optimization and radiometric analysis, a uniform ring is often useful for visualization. This DLL can be used with a Source DLL in non-sequential mode to provide a uniformly spaced ring of rays. The usual position and tilts parameters can be used to reorient a Ring as desired.

Sample files

- Ring.zar demonstrates the use of Ring.dll.

Downloads

- Ring.dll must be placed in the Zemax/DLL/Source directory. This can be accomplished by opening Ring.zar.
- Ring.cpp is the C++ source code.

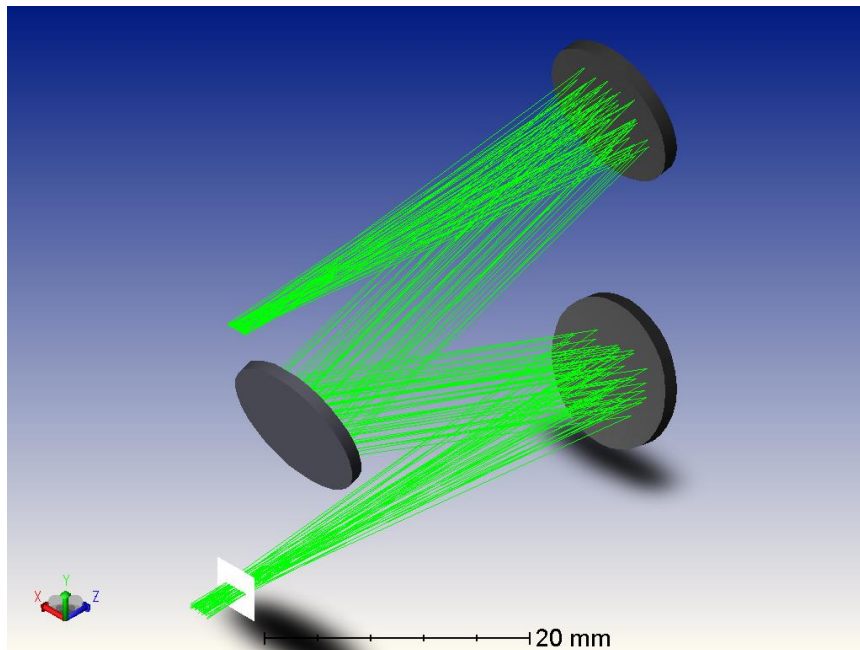
In this article

- Introduction
- Implementation
- Installing Ring.dll
- Instructions for use with a Source DLL object
- Examples

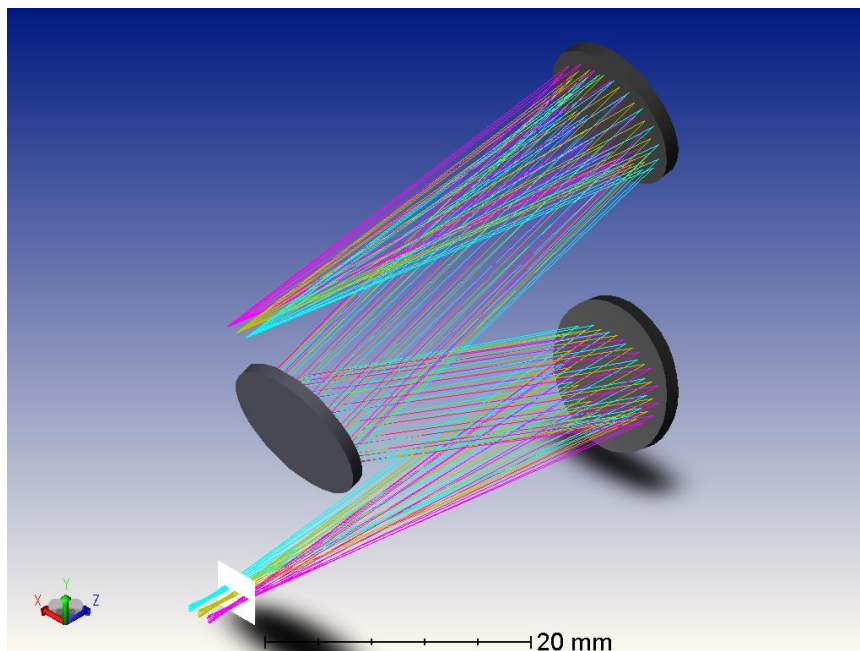
Introduction

For both radiometric analysis (for example determining the power incident on a detector) and optimization, the best choice for a source is almost always a pseudo-random source such as a Source Ellipse, with enough analysis rays to provide for smooth statistics. Frequently, this means tens of thousands to millions of rays. For visualization, OpticStudio provides the capability to designate a lesser number of Layout Rays to appear in layout plots. These layout rays are still randomly distributed. If only a few rays are used, they generally do not well represent the design. If enough rays are used to visualize the extent of the volume occupied by rays, then the plot can become crowded.

Below, a monochromator layout is visualized using the same pseudo-random sources (Source Two Angle) as used for radiometry, but with a reduced number of layout rays.



A ring provides better visualization.



One can, of course, generate a ray list to be used with a Source File object. Many users do that with tools like Excel, Matlab, or Mathematica. But this is time consuming, and eventually one winds up with a large number of such files, each with some unique but still forgettable name.

Ring.dll, when used as the data for a Source DLL object, provides the ability to generate a uniform ring of rays where the parameters defining the ring are entries in the Non-sequential Component Editor (NCE).

In this article, we will show how to use Ring.dll in OpticStudio's Non-sequential Mode.

Implementation

Code defining the Ring.dll was written in C++ and compiled with Microsoft Visual Studio 2019. The DLL implements two functions:

- UserParamNames -- receives a pointer to an array. The first element in the array contains an integer indicating the number of the user-defined parameter whose name should be returned by the function. The name is returned by modifying the elements of the same array. This allows the DLL to provide the column headings for the user-defined parameters.
- UserSourceDefinition – receives a pointer to an array. The array contains the values of the user-defined parameters which have been entered in the NCE. For each call, the function returns the parameters defining a single ray. These are the values for x, y, z, l, m, n and intensity. The values are returned by modifying the passed array.

Notice that the paradigm used for calling the source DLL expects to make a separate call for each ray. This works well if they are served from a random distribution. But in this case, we want to generate and serve a specific set of rays to form an evenly spaced ring. To provide for this a special approach is taken. On first call, and on any call for which the user defined parameters have changed, the complete ray set is calculated, and a ray counter is reset. Each call then returns the ray designated by the counter and then increments the counter. The parameters passed to UserSourceDefinition do not include “# Layout Rays” so the number of rays in the ring must be defined separately in the user-defined parameters. If the number of layout rays is less than the number of rays in the ring, then only a portion of the ring will be shown. If the number of layout rays is more than the number of rays in the ring, then some rays will be duplicated. (This is also true for analysis rays, if any are defined.)

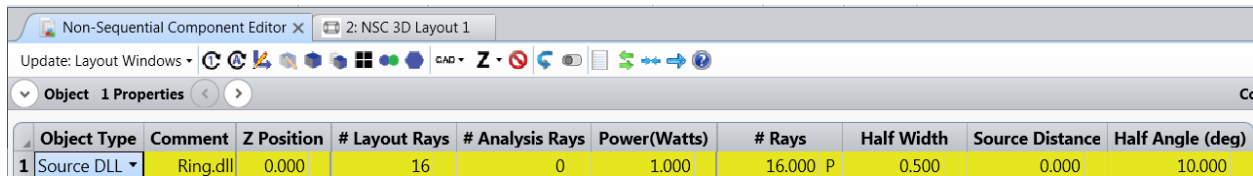
In most cases, the ring will be used for visualization only, and contain only layout rays, and so will contribute zero power to analyses. In this case, a good practice is to define the number of rays in the “# Layout Rays” entry and use a pickup in the “# Rays” entry to control the ring.

Installing Ring.dll

The file Ring.dll should be placed in the Zemax DLL/Sources directory. In the default installation that resides in the user documents directory. Opening the example file Ring.zar will place Ring.dll in the correct directory automatically.

Instructions for use as a Source DLL

Below is a section of the NCE in which a Ring has been defined. (Columns not related to the discussion have been hidden.)



The screenshot shows the 'Non-Sequential Component Editor' window with a tab for '2: NSC 3D Layout 1'. Below the toolbar, the 'Object 1 Properties' tab is active, displaying a table of parameters for a 'Source DLL' object.

Object Type	Comment	Z Position	# Layout Rays	# Analysis Rays	Power(Watts)	# Rays	Half Width	Source Distance	Half Angle (deg)
1 Source DLL	Ring.dll	0.000	16	0	1.000	16.000 P	0.500	0.000	10.000

- The source has been entered as a Source DLL and Ring.dll selected from the pull-down list.
- The number of rays in the ring has been entered as the # Layout Rays.
- There are zero analysis rays, so the Power of 1 Watt does not affect results.
- The # Rays in the ring implements a pickup from # Layout Rays.
- The Half Width of the ring is 0.5 lens units.
- With a Source Distance of 0, the Half Angle controls the ring. (to be explained)

Parameters

Four parameters control the generation of rays for the ring:

Rays: Sets the number of rays.

- If less than 2, then 2 rays are generated.
- If greater than 101, then 101 rays are generated.
- #Rays does not control the number of rays for display or analysis.
- If these are fewer, then only a portion of the rays are used.
- If greater, then some rays are duplicated.

Half Width: Controls the extent of the ring in position space.

- With respect to the source location, all rays have an origin on the local x-y plane ($z = 0$).
- Rays will originate from xy locations forming an evenly spaced ring of radius Half Width centered on the origin.
- The ring begins on the x-axis with the first ray origin at $x = \text{Half Width}$ and $y = 0$.

Source Distance: If non-zero, rays appear to emanate from a point on the z axis at this distance from the location of the source object. In that case, the Half Angle is ignored.

- A negative value results in diverging rays.
- A positive value results in converging rays.

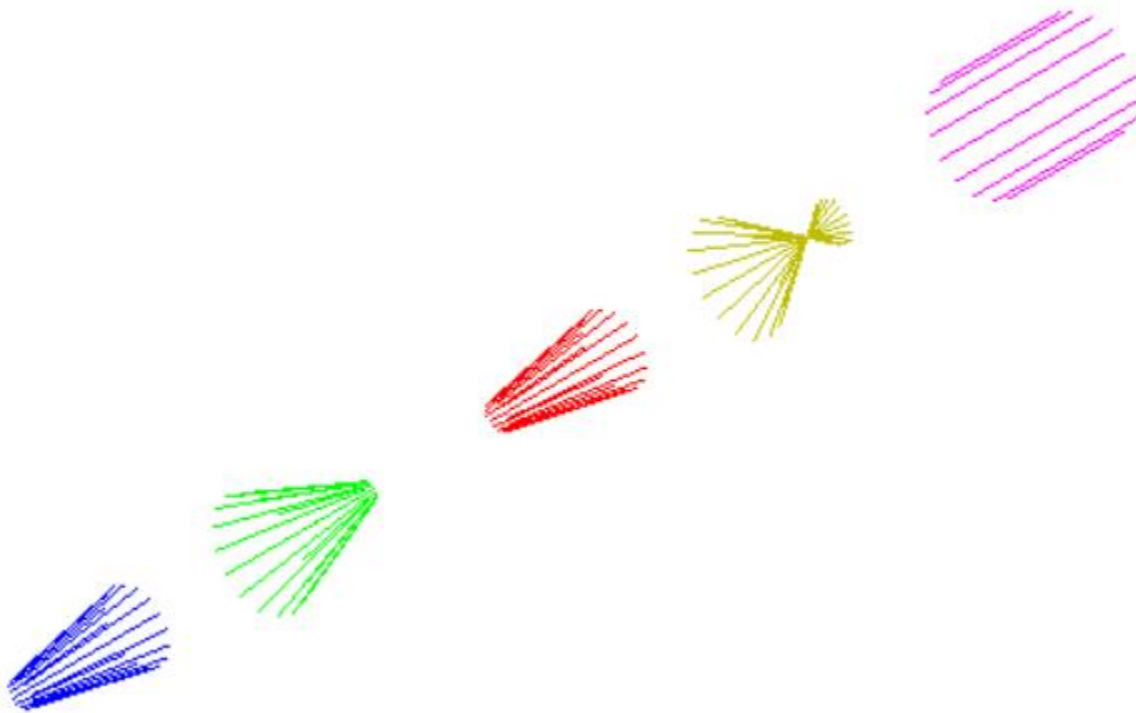
Half Angle: Controls the angular divergence of the ring in degrees.

- Each ray is directed from its origin within a plane containing the source origin, ray origin, and the z-axis. The set of rays forms a diverging or converging cone.

Examples

Below is an image of the NCE from the example file Ring.zar and a layout of the resulting rings.

Non-Sequential Component Editor x 1: NSC 3D Layout 1											
Update: Layout Windows											
Object 1 Properties											
Object Type	Comment	Z Position	# Layout Rays	# Analysis Rays	Power(Watts)	# Rays	Half Width	Source Distance	Half Angle (deg)		
1 Source DLL	Ring.dll	0.000	16	0	1.000	16.000 P	0.500	0.000	10.000		
2 Source DLL	Ring.dll	10.000	16	0	1.000	16.000 P	2.000	0.000	-20.000		
3 Source DLL	Ring.dll	20.000	16	0	1.000	16.000 P	0.500	-3.000	10.000		
4 Source DLL	Ring.dll	30.000	16	0	1.000	16.000 P	2.000	3.000	10.000		
5 Source DLL	Ring.dll	40.000	16	0	1.000	16.000 P	2.000	0.000	0.000		



The NCE entries from top to bottom produce the rings depicted from left to right as follows:

1. A diverging cone with Half Angle 10 degrees;
2. A converging cone with Half Angle -20 degrees;
3. A diverging cone with rays appearing to emanated from 3 lens units behind the source (notice the Half Angle is ignored);
4. A converging cone with rays converging to a point 3 lens units beyond the source;
5. A collimated cone with Half Width 2 lens units and Half Angle zero.

**** End ****